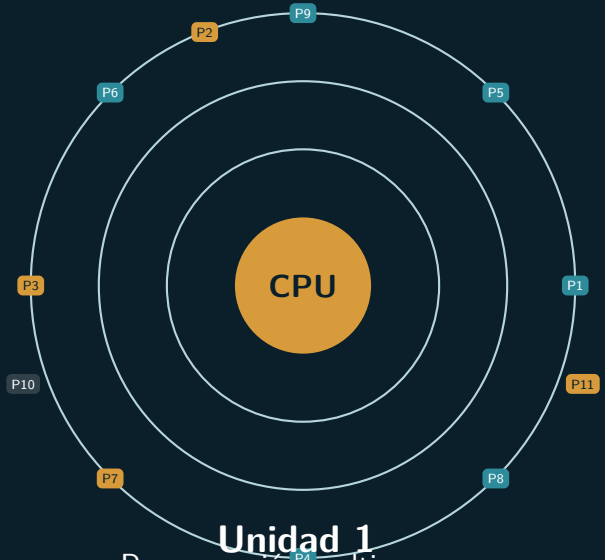


Unidad 1

Programación multiproceso

Programación de servicios y procesos

Material basado en la obra de José María Pérez Puentes ▪ Licencia CC BY-NC-SA 4.0

Contenido

Del concepto de proceso a su gestión desde Java: fundamentos, planificación, concurrencia y casos prácticos.

1. **Fundamentos**
   Programa, proceso y servicio
2. **Planificación y multitarea**
   Planificador, tipos de sistemas y reparto de la CPU
3. **Concurrencia**
   Hilos frente a procesos; programación paralela y distribuida
4. **Ciclo de vida y cooperación**
   Estados, comunicación (IPC) y sincronización
5. **Multiproceso en Java**
   `Process`, `ProcessBuilder`, flujos de E/S y casos prácticos

Programa, proceso y servicio

Tres conceptos que conviene no confundir

Programa

El objeto estático

- Conjunto de instrucciones que se pueden ejecutar directamente en una máquina.
- Se almacena normalmente en un fichero binario, en un medio de almacenamiento secundario.

Proceso

La entidad dinámica

- Instancia de un programa en ejecución.
- Para ejecutarse utiliza memoria, procesador y dispositivos de E/S.

Servicio

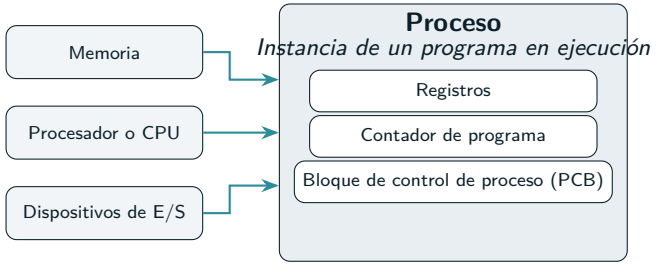
El proceso que atiende a otros

- Proceso particular que presta servicio a otros procesos, del mismo equipo o de otros en red.
- Se ejecuta en segundo plano, de forma continua y sin uso directo del usuario; lo inicia el sistema operativo en el arranque.

Ejecutar un programa significa crear un proceso; un servicio es un proceso que permanece activo para atender a otros.

Anatomía de un proceso

Recursos que consume y elementos que lo describen



Observar procesos

Linux
`ps  ps -f  ps -Af`
macOS
`ps`
Windows
`tasklist`
Administrador de tareas

El número de procesos simultáneos supera ampliamente el número de núcleos de la CPU. Corresponde al sistema operativo repartir los recursos de forma ordenada, justa y eficiente.

Planificación de procesos

El planificador reparte los recursos entre los procesos que los demandan

Objetivos del planificador

- **Maximizar** el rendimiento del sistema.
- **Maximizar** la equidad en el reparto.
- **Minimizar** los tiempos de espera.
- **Minimizar** los tiempos de respuesta.

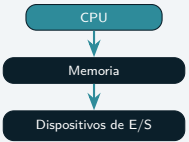
Niveles de planificación

- **Largo plazo**: planificador de trabajos. Decide qué procesos son admitidos para su ejecución.
- **Corto plazo**: planificador de procesador. Reparte el tiempo de procesador entre todos los procesos.
- **Medio plazo**: gestión de memoria. Controla el paso de procesos de la memoria principal a la secundaria (suspensión) y viceversa (reanudación).

Sistemas según el número de procesadores

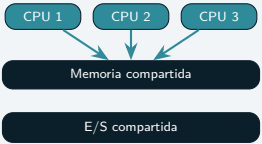
De un único procesador a sistemas con memoria compartida o independiente

Monoprocesador



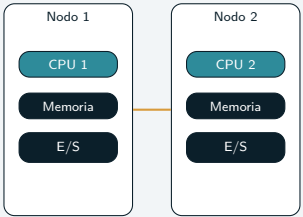
La ejecución concurrente de varios procesos se conoce como **multiprogramación**.

Multiprocesador fuertemente acoplado



Memoria y dispositivos de E/S compartidos entre todos los procesadores.

Multiprocesador débilmente acoplado



Cada procesador posee su propia memoria y sus dispositivos de E/S.

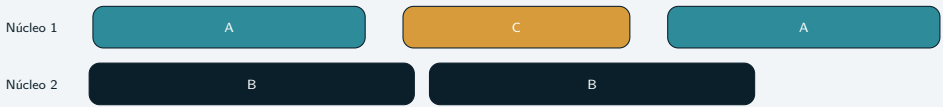
Multitarea

Varios procesos comparten el procesador a lo largo del tiempo

Un único núcleo · tiempo compartido



Dos núcleos · paralelismo real



- La multitarea crea la ilusión de simultaneidad alternando los procesos en la CPU en cortos intervalos.
- Mientras un proceso espera una operación de E/S, el procesador ejecuta otro y aprovecha mejor sus recursos.

Hilos y procesos

El hilo es la unidad más pequeña de procesamiento dentro de un proceso



Procesos

- Constan de uno o más hilos.
- Son independientes unos de otros.
- Los gestiona el sistema operativo.
- Se comunican a través del SO.

Hilos

- Un hilo siempre existe dentro de un proceso.
- Comparten los recursos del proceso de forma directa.
- Los gestiona el propio proceso.
- La comunicación la controla el proceso.

Programación concurrente

Procesos concurrentes: los que se ejecutan simultáneamente durante un intervalo de tiempo.
La simultaneidad puede ser real (sistemas multiprocesador) o simulada (un mismo procesador).
Programación concurrente es también el conjunto de técnicas para que varios procesos cooperen en una tarea.

Ventajas

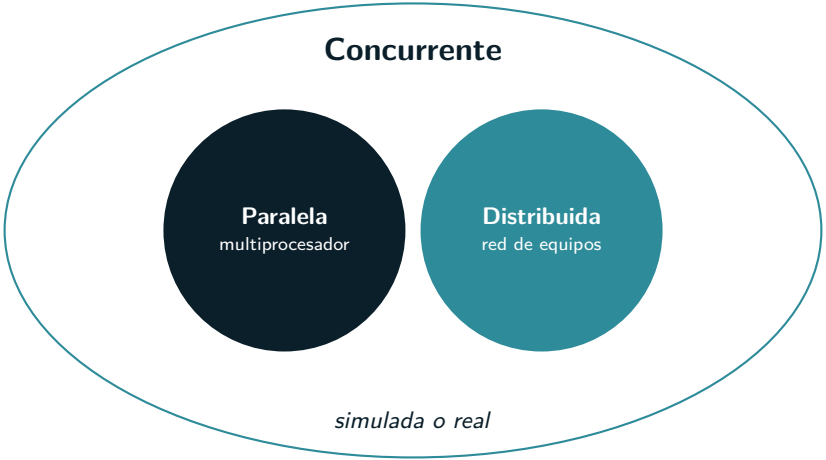
- Incrementa el rendimiento: el procesador pasa a otro proceso cuando el actual está desocupado o espera una operación de E/S.
- Reduce el tiempo de respuesta, porque el tiempo de procesador se distribuye entre todos los procesos en ejecución.

Inconvenientes

- Dificulta implementar mecanismos adecuados de sincronización y comunicación entre procesos.
- La sobrecarga puede anular la mejora de rendimiento que aporta la concurrencia.
- En sistemas distribuidos, la red puede convertirse en cuello de botella si no soporta el tráfico de mensajes.

Concurrente, paralela y distribuida

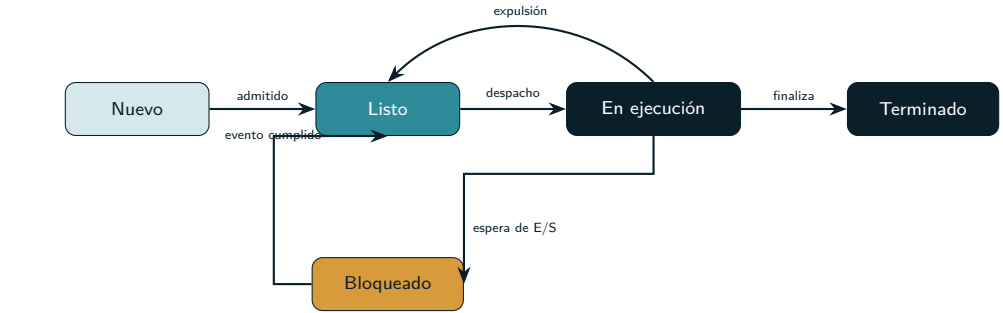
Tres formas de ejecutar varios procesos a la vez



Concurrente	Paralela	Distribuida

Estados de un proceso

Ciclo de vida desde su creación hasta su finalización



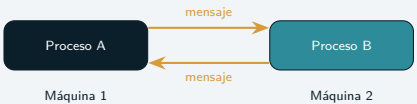
- **Nuevo**: recién creado, con recursos iniciales; espera ser admitido.
- **Listo**: en memoria principal, a la espera de CPU.
- **En ejecución**: usa la CPU; uno por núcleo a la vez.
- **Bloqueado**: espera un evento, como una operación de E/S.
- **Terminado**: libera sus recursos; queda registrado su estado final.

Solo un proceso puede estar en ejecución por núcleo y unidad de tiempo. Cuando se cumple la condición de

Comunicación entre procesos (IPC)

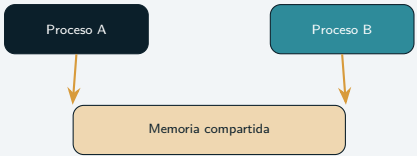
Dos maneras de comunicarse y cinco alternativas para hacerlo

Intercambio de mensajes



Forma habitual cuando los procesos están en máquinas distintas. La información viaja según un protocolo establecido previamente.

Recursos compartidos



Solo se puede usar si ambos procesos están en la misma máquina; permite sincronizar según el valor o estado del recurso.

Alternativas para implementar la IPC: Sockets · **Flujos de entrada y salida** (es la que se usa en los ejemplos de esta unidad) · RPC · Sistemas de persistencia · Servicios a través de...

Sincronización entre procesos

Cuatro mecanismos para gestionar el flujo de trabajo y su equivalente en Java

Ejecución

Ejecutar procesos desde un proceso
`ProcessBuilder.start()`

Espera

Bloquear un proceso hasta que otro termine
`Process.waitFor()`

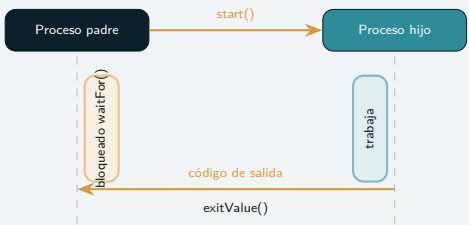
Generación de código de terminación

Indicar cómo ha terminado la ejecución
`System.exit(código)`

Obtención de código de terminación

Recuperar el código de otro proceso
`Process.exitValue()`

En la práctica



Por convención, el código 0 indica una terminación correcta.

Multiproceso en Java

Dos formas de crear procesos y una clase para controlarlos

java.lang.Runtime

Interactúa con el JRE: lanzar procesos, invocar al recolector de basura, consultar la memoria libre. Cada aplicación tiene una única instancia, accesible con `Runtime.getRuntime()`.

start()

java.lang.ProcessBuilder

Gestiona los atributos del proceso antes de lanzarlo: comando, directorio de trabajo, entorno y redirecciones de E/S.

Las clases en detalle

Métodos principales de `Process` y de `ProcessBuilder`

Process

`destroy()`	Destruye el proceso
`exitValue()`	Valor de retorno al finalizar
`getErrorStream()`	`InputStream` de la salida de error
`getInputStream()`	`InputStream` de la salida normal
`getOutputStream()`	`OutputStream` de la entrada normal
`isAlive()`	Indica si sigue en ejecución
`waitFor()`	Espera a que el proceso termine

ProcessBuilder

`start()`	Inicia un nuevo proceso con los atributos dados
`command()`	Obtiene o asigna programa y argumentos
`directory()`	Obtiene o asigna el directorio de trabajo
`environment()`	Información del entorno de ejecución
`redirectError()`	Destino de la salida de errores
`redirectInput()`	Origen de la entrada estándar
`redirectOutput()`	Destino de la salida estándar

Lanzar un proceso con ProcessBuilder

El constructor recibe el comando y sus argumentos

```
// Windows
ProcessBuilder pb = new ProcessBuilder("cmd", "/C", "dir");

// Linux
ProcessBuilder pb = new ProcessBuilder("/bin/bash", "-c", "ls");

// macOS
ProcessBuilder pb = new ProcessBuilder("sh", "-c", "ls");

Process p = pb.start();
```

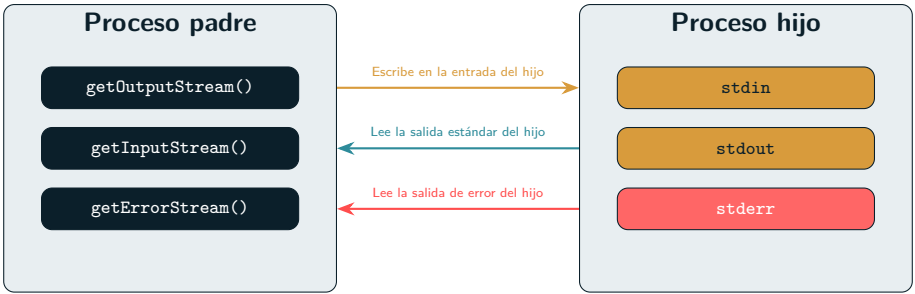
Constructores disponibles

```
ProcessBuilder(List<String> command)
ProcessBuilder(String... command)
```

- **Construir**: se indica el comando y los argumentos.
- **Configurar**: directorio, entorno y redirecciones.
- **Iniciar**: `start()` lanza el proceso.
- **Controlar**: el objeto `Process` permite gestionarlo.

Gestión de la E/S de un proceso

Un subproceso no tiene consola: el padre accede a sus flujos estándar



Si se llama a `redirectErrorStream(true)`, la salida de error llega junto a la estándar por `getInputStream()` y no hace falta tratarla por separado. Para trabajar cómodamente con los flujos se envuelven con decoradores como `BufferedReader` y `PrintWriter`.

Leer y escribir en un proceso hijo

Decoradores sobre los flujos del proceso

Lectura

```
Process p = pbuilder.start();
BufferedReader processOutput = new
  BufferedReader(new InputStreamReader(
    p.getInputStream()));
String linea;
while ((linea = processOutput
        .readLine()) != null) {
  System.out.println("> " + linea);
}
processOutput.close();
```

`InputStreamReader` adapta el flujo de bytes a caracteres y `BufferedReader` permite leer línea a línea. `readLine()` devuelve `null` cuando no quedan líneas.

Escritura

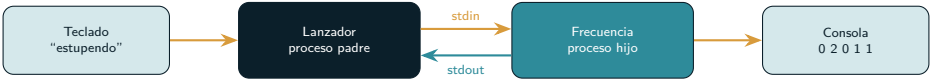
```
PrintWriter toProcess = new
  PrintWriter(new BufferedWriter(
    new OutputStreamWriter(
      p.getOutputStream(), "UTF-8")),
    true);

toProcess.println("Enviar al hijo");
```

`PrintWriter` ofrece métodos parecidos a `System.out.print` y, con el autoflush activado, envía cada línea al hijo al instante.

Caso práctico · Comunicación entre procesos

Lanzador ejecuta Frecuencia y le envía un texto por su entrada estándar



Proceso hijo · Frecuencia.java

```
Scanner scanner = new
  Scanner(System.in);
String input = scanner.nextLine();
for (char c : input.toLowerCase()
      .toCharArray()) {
  switch (c) {
    case 'a': countA++; break;
    case 'e': countE++; break;
    // ... i, o, u
  }
}
System.out.println(countA + " " +
  countE + " " + ...);
```

Proceso padre · Lanzador.java

```
String cp = System.getProperty(
  "java.class.path");
ProcessBuilder pb = new
  ProcessBuilder("java", "-cp", cp,
    "Frecuencia");
Process process = pb.start();

writer.write(inputString);
writer.newLine();
writer.flush();
writer.close();

String line = processOutputReader
  .readLine();
process.waitFor();
```

`newLine()` y `close()` marcan el final de la entrada: sin ellos, `Frecuencia` quedaría esperando datos.

Redirección de la E/S

Se configura en `ProcessBuilder` antes de iniciar el proceso

Métodos de redirección

`redirectOutput(File)`
salida estándar

`redirectInput(File)`
entrada estándar

`redirectError(File)`
salida de error

También admiten objetos `ProcessBuilder.Redirect` con cuatro comportamientos. Es la vía habitual para guardar resultados en ficheros de log o de errores.

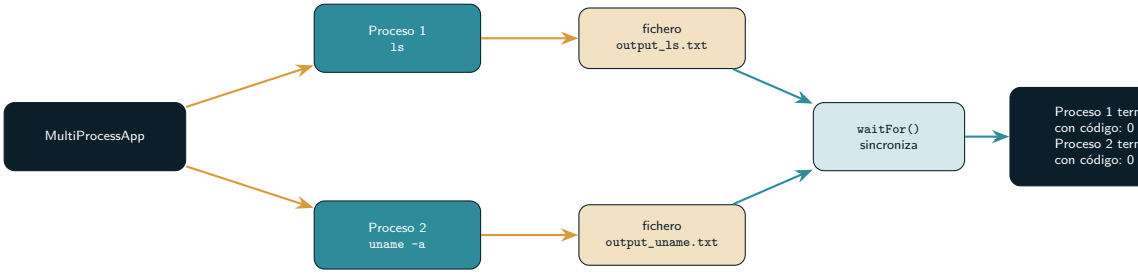
Valor de Redirect

`Redirect.DISCARD`	La información se descarta.
`Redirect.to(File)`	Se guarda en el fichero; si existe, se vacía.
`Redirect.from(File)`	La información se lee del fichero indicado.
`Redirect.appendTo(File)`	Se añade al fichero; si existe, no se vacía.

```
pb.redirectOutput(new File("salida.txt"));
```

Caso práctico · Redirección y sincronización

Dos procesos en paralelo con su salida guardada en ficheros distintos




```
ProcessBuilder pb1 = new ProcessBuilder("sh", "-c", "ls");
pb1.redirectOutput(new File("output_ls.txt"));
Process process1 = pb1.start();   // idem con uname -a

int exitCode1 = process1.waitFor();
int exitCode2 = process2.waitFor();
```

Ambos procesos arrancan sin esperarse y trabajan a la vez; las llamadas a `waitFor()` los sincronizan y recogen sus códigos de salida.

Ideas clave

Lo esencial de la unidad para diseñar aplicaciones que lancen y coordinen procesos.

- **Un proceso es un programa en ejecución.** Consume memoria, CPU y dispositivos de E/S.
- **El sistema operativo planifica.** Reparte los recursos buscando rendimiento, equidad y menores esperas.
- **La concurrencia tiene un coste.** Mejora el rendimiento, pero exige comunicación y sincronización.
- **ProcessBuilder y Process.** Se configura, se lanza con `start()` y se controla con `waitFor()` y `exitValue()`.
- **Los flujos conectan padre e hijo.** `getInputStream`, `getOutputStream` y las redirecciones gestionan la E/S.